Day 32 – PM session

Sudhanshu Biswas

Part C: Interview Ready

Q1 — 1000 Trees vs 100 Trees

No — you shouldn't default to 1000 trees just because accuracy is the same. The tradeoffs:

- Compute cost: 10× more trees = 10× longer training. For 3000 features or large datasets this is non-trivial.
- Prediction latency: Each inference traverses all trees. 1000 trees will be slower at serving time — critical for real-time APIs.
- Marginal improvement: RF accuracy saturates after ~100–200 trees. Beyond that, each tree adds diminishing returns on variance reduction.
- Memory: 1000 trees require ~10× RAM for model storage and serving — relevant for containerised deployment (e.g. Lambda, small pods).

Use the elbow on an n_estimators vs OOB error curve to pick the smallest tree count where error has plateaued.

Q2 — compare_models()

```
import time

import numpy as np

import pandas as pd

from sklearn.model_selection import cross_validate


def compare_models(X, y, models_dict):

    results = []

    scoring = ['accuracy', 'f1_weighted']


    for name, model in models_dict.items():

        t0 = time.time()

        cv = cross_validate(

            model, X, y, cv=5, scoring=scoring,

            return_train_score=False, n_jobs=-1

        )

        elapsed = time.time() - t0
```

```
    results.append({
        'Model': name,
        'Accuracy_mean': round(cv['test_accuracy'].mean(), 4),
        'Accuracy_std':  round(cv['test_accuracy'].std(), 4),
        'F1_mean':       round(cv['test_f1_weighted'].mean(), 4),
        'F1_std':        round(cv['test_f1_weighted'].std(), 4),
        'Train_time_s':  round(elapsed, 2)
    })

  return pd.DataFrame(results).set_index('Model')
```

Q3 — Feature Importance Instability Debug

```
rf1 = RandomForestClassifier(n_estimators=10).fit(X_train, y_train)
rf2 = RandomForestClassifier(n_estimators=10).fit(X_train, y_train)
# feature_importances_ are completely different!
```

Root cause: no random_state set. RandomForestClassifier uses randomness at two levels — bootstrap sampling per tree and feature subsampling at each split. With only 10 trees and no seed, each run produces a different subset of trees, so MDI-based feature importances vary wildly between runs. Fix:

```
rf1 = RandomForestClassifier(n_estimators=10, random_state=42).fit(X_train, y_train)
rf2 = RandomForestClassifier(n_estimators=10, random_state=42).fit(X_train, y_train)
# Now feature_importances_ are identical

# Additional note: even with a seed, 10 trees is too few.
# Use >= 100 trees for stable importance estimates.
```

Best practice: always set random_state for reproducibility, and use permutation importance over MDI for more reliable rankings on correlated features.